University of Mascara
Department of Computer sciences
Second Year Engineer

Algorithmic and Complexity

Problem Set 2

Problem 1

Write two functions to find a specific integer in an array (two cases: sorted and unsorted array), then find their complexities (worst and best cases).

Problem 2

Which algorithm is faster: Bubble Sort or Selection Sort?

Problem 3

How does merge sort algorithm rearrange the elements in the following array to achieve a sorted order?

A= {64, 34, 25, 12, 22, 11, 90, 45, 71, 8}

Problem 4

Given a Selection Sort code, analyze it for unnecessary swaps. Optimize the code to minimize the number of swaps while ensuring the correctness of the sorting algorithm.

Problem 5

Counting sort is a non-comparison-based sorting algorithm that counts the occurrences of each distinct element in the input array. It then uses this count to determine the positions of the elements in the sorted output. Counting sort is particularly efficient when the range of input values is not significantly larger than the number of values to be sorted.

```
void countingSort(int arr[], int n) {
    int max = arr[0];

// Find the maximum value in the array
    for (int i = 1; i < n; i++) {
        if (arr[i] > max) {
                    max = arr[i];  }    }
```

```
//Create a count array initialized to 0
int *count=(int *)calloc(max+1,sizeof(int));

// Count the occurrences of each element
    for (int i = 0; i < n; i++) {
        count[arr[i]]++;    }

// Reconstruct the sorted array
    int index = 0;
    for (int i = 0; i <= max; i++) {
        while (count[i] > 0) {
            arr[index++] = i;
            count[i]--;        }    }

// Free the allocated memory
    free(count);}
```

1) What are the time and space complexities of this counting sort algorithm?

2) Improve the function to sort an array containing **N** elements where only **k** distinct values are present. In this scenario, **N** is very large, while **k** is a small number.

3) Analyze and discuss the time and space complexities of the new solution.

Problem Set 2

Problem 7

1. What does the following function do?

```
void swap(int* a, int* b) {
    int temp = *a; *a = *b; *b = temp;}

void cocktailSort(int A[], int n) {
int start = 0;
int end = n - 1;
int swapped = 1;

while (swapped) {
  swapped = 0;

  for (int i = start; i < end; i++) {
          if (A[i] > A[i + 1]) {
              swap(&A[i], &A[i + 1]);
              swapped = 1;
          }
  }
  end--;

  for (int i = end; i >= start; i--) {
          if (A[i] > A[i + 1]) {
              swap(&A[i], &A[i + 1]);
              swapped = 1;      }   }
  start++;
  }
}
```

2. Calculate its complexities (Best and worst)

3. Can you improve the given code to make it faster ?

Problem 8

Same questions:

```
#include <stdio.h>
#include <stdlib.h>

int partition(int A[], int low, int
high) {
    int pivot = A[high];
    int i = (low - 1);

    for (int j = low; j < high; j++) {
        if (A[j] <= pivot) {
            i++;
            // Swap A[i] and A[j]
            int temp = A[i];
            A[i] = A[j];
            A[j] = temp;
        }
    }

    // Swap A[i+1] and A[high]
    int temp = A[i + 1];
    A[i + 1] = A[high];
    A[high] = temp;

    return (i + 1);
}

int quickSelect(int A[], int low, int
high, int k) {
    if (low == high) {
        return A[low];
    }

    int pivotIndex = partition(A, low,
high);

    if (k == pivotIndex) {
        return A[pivotIndex];
    } else if (k < pivotIndex) {
        return quickSelect(A, low,
pivotIndex - 1, k);
    } else {
        return quickSelect(A, pivotIndex
+ 1, high, k);
    }
}

int main() {
 int A[] = {3, 2, 1, 5, 4};
 int n = sizeof(A) / sizeof(A[0]);
 scanf("%d", k)

 if (k <= 0 || k > n) {
        printf("Invalid K value.\n");
        return 1;
    }

 int r = quickSelect(A, 0, n-1, k-1);
 printf("The element: %d\n", r);

    return 0;
}
```